Ancient Machine 2 (Solution)

Author : Tomohito Hoshii

Introduction

In this review, we only explain solutions which lead to full score solutions. However, there are other solutions which give good results but do not lead to full score solutions.

In the following, the indices of elements of arrays are zero-based (i.e. they start from 0), and "the value set in the memory area of the machine" is called "the state" of the machine.

$M = 1002$ (10 points)

We can determine the i-th character by $m = i + 3$. Concretely, we construct the sequences as follows.

$$(a_j, b_j) = \begin{cases} (j+1, j+1) & 0 \leq j \leq i-1 \\ (i+1, i+2) & j = i \\ (j, j) & j = i+1, i+2 \end{cases}$$

The i-th character is '`0`' if the final state is $i + 1$, and it is '1' if the final state is $i + 2$.

Since $N = 1000$, we determine each character one-by-one from the first one, and get a solution with $M = 1002$.

$M = 1001$ (10 points)

For the $|T|$ characters from the last one, it is possible to determine whether the string S coincides with the string T or not by $m = |T| + 1$.

For example, if $T =$"`101`", we construct as follows.

- $a = [0, 2, 0, 2]$
- $b = [1, 1, 3, 1]$

The current state is i if the characters are the same until the $(i - 1)$-th character of T.

If the final state is $|T|$, the last characters coincide with T; otherwise, the last characters do not coincide with T.

By this method, we determine the characters from the last one, and get a solution with $M = 1001$.

$M = 502$ (37 points)

We use an $M = 1002$ solution as above for the first half of characters (500 characters), and use an $M = 1001$ solution as above for the last half of characters (500 characters). Combining them, we get a solution with $M = 502$.

$M = 114$ (97 points)

We can achieve $M = 114$ using linear algebra.

There are 1000 unknown quantities. We would like to have 1000 independent linear equations in 1000 variables. Since we know each quantity is either 0 or 1, we can consider equations mod 2.

Concretely, we proceed as follows.

We get the sum of S_i's (mod 2) for every i with $i \equiv q \pmod{p}$ by $m = 2p$. Here we choose the values of p and q by row reduction (or Gaussian elimination) so that the vectors for chosen p and q (for example, $[0, 1, 0, 0, 1, 0, 0, 1, 0, \dots]$ if $p = 3$ and $q = 1$) are linearly independent.

We can find 1000 linearly independent vectors in the range $p \leq 57$. Hence we get a solution with $M = 114$.

$M = 102$ (100 points)

Combining an $M = 114$ solution and an $M = 502$ solution, we achieve $M = 102$ as follows.

To get information on a particular character amounts to choose the vector $[0, 0, \dots, 0, 1, 0, \dots, 0]$.

Up to 102, we determine the first 100 characters and the last 101 characters; hence we get 201 vectors. Additionally, in the range $p \leq 51$, we get 799 vectors such that the $201 + 799 = 1000$ vectors are linearly independent. Hence we get a solution with $M = 102$.

Cell Automaton

Author : Akihito Yoneyama

Subtask 1

Following the rules in the problem statement, it is enough to calculate the color of each cell (x, y) in the range $-100 \leq x, y \leq 100$ at time t $(0 \leq t \leq 50)$.

Subtask 2

A cell is black if the shortest distance to initially black cells is t; it is gray if the shortest distance to initially black cells is $t - 1$; otherwise, it is white. Therefore, it is enough to calculate the shortest distance for every cell (x, y) in the range $-2000 \leq x, y \leq 2000$ by Breadth-first search (BFS).

Proof

We prove the assertion by induction on t.

The assertion is true if $t = 0$.

If the assertion is true in time t,

- a cell whose shortest distance is $d > t + 1$ is white in time $t + 1$ because it is initially white and it is not adjacent to a currently black cell (whose shortest distance is t);
- a cell whose shortest distance is $d = t + 1$ is black in time $t + 1$ because it is initially white and it is adjacent to a currently black cell (whose shortest distance is t);
- a cell whose shortest distance is $d = t$ is gray in time $t + 1$ because it is initially black;
- a cell whose shortest distance is $d = t - 1$ is white in time $t + 1$ because it is initially gray;
- a cell whose shortest distance is $d < t - 1$ is white in time $t + 1$ because it is initially white and it is not adjacent to a currently black cell (whose shortest distance is t).

Therefore, the assertion is true also in time $t + 1$.

Subtask 3

Sorting the X-coordinates in ascending order, we classify the relations between adjacent cells into the following types and its contribution the answer:

- It does not clash yet.
- It has just clashed (if the difference of the X-coordinates is even).
- It already clashed.

The total time complexity is $O(QN \log N)$.

Subtask 4

In the classification for Subtask 3, the contribution of each type to the answer is a linear function in t. Thus, pre-calculating the time when the classification changes and the amount of change of the linear function in advance, we can calculate the answer by sweeping t in ascending order and updating the linear function representing the current answer. The total time complexity is $O(N \log N + Q)$.

Subtask 5

We calculate the contribution of each edge of a square to the answer. For each edge, we have a restriction to a remaining square $(X_i + p, Y_i + q)$ from other squares such as "$|p| \leq a$ after time t_0" or "$|q| \leq a$ after time t_0."

For each edge, calculating these restrictions from each square, we can determine the number of cells (contribution to the answer) satisfying the conditions at time t. The total time complexity is $O(QN^2)$.

Subtask 6

The contribution of each edge to the answer is a linear function in t on an interval whose boundaries are the time when the restriction is added (t_0 in the restriction in Subtask 5) and the time when the number of cells satisfying the conditions becomes 0. Therefore, pre-calculating these time and the amount of change of the linear function in advance, we can calculate the answer by sweeping t in ascending order and updating the linear function representing the current answer. The total time complexity is $O(N^2 + Q)$.

Full Score Solution

First, we consider the case where there is no pair of squares such that $x, y, x + y, x - y$ are equal.

The linear function in t representing contribution of each edge to the answer changes only in the following situation:

- When a corner of a square clashes another square and vanishes, the edges of the squares are reduced.
- When an edge vanishes completely because it is reduced by other squares from both sides, the edges of the squares clash and are reduced.

Both of these cases occur at most $O(N)$ times.

For the first case, we can enumerate them in $O(N \log N)$ time by sweeping the plane about 8 times using segment trees. For the second case, we can enumerate them using the results for the first case for each edge.

By a similar process, we can calculate the answer even when there is a pair of squares such that $x, y, x+y, x-y$ are equal. The total time complexity is $O(N \log N + Q)$.

Garden (Solution)

Author : Yuto Watanabe

Overall consideration

If a cell is represented by (x, y), the values of x and y are called the x-coordinate and the y-coordinate, respectively. The values of a, b, c, d, which determine the position of the garden, are called the left end, the right end, the upper end, and the lower end, respectively.

The artworks placed on the cells (x, y), $(x + d, y)$ are the same, and the artworks placed on the cells (x, y), $(x, y + d)$ are the same. So, there is no reason to make a garden whose horizontal width exceeds d or vertical height exceeds d. Therefore, it is enough to consider the problem in the 2-dimensional torus in the range $0 \leq x < d, 0 \leq y < d$ (namely, we consider $x = 0$ and $x = d - 1$ are adjacent from left to right, and $y = 0$ and $y = d - 1$ are adjacent from down to top). Then, the conditions for artworks of type A become $a \leq P_i \leq b$ and $c \leq Q_i \leq d$, and the conditions for artworks of type B become $a \leq R_j \leq b$ or $c \leq S_j \leq d$.

Subtask 1 (15 points)

Since it is bothering if the conditions contain the logical operator "or", we would like to remove them. For every artwork of type B, we check whether it satisfies one of $a \leq R_j \leq b$ or $c \leq S_j \leq d$. Then the conditions on the garden become a conjunction of conditions connected by the logical operator "and". Therefore, for each x,y-coordinate, we need to solve the following problem (we call it Problem X):

- Assume that the integers between 0 and $d - 1$ are placed on a circle. We choose an interval on a circle so that some of the integers should be contained in the interval. Calculate the minimum length of the interval.

Problem X can be solved in time $O(D)$. Combining with the time to check every artwork, the total time complexity is $O(N + 2^M D)$.

Subtask 2 (6 points)

We check all the left ends, the right ends, the upper ends, and the lower ends. For every artwork among the $N+M$ artworks, we need to check whether it satisfies the conditions. The total time complexity is $O(D^4(N+M))$.

Subtask 3 (8 points)

If we fix the left end, the right end, and the upper end, we need to consider the condition of the form "the lower end should be greater than $*$" for every artwork among the $N + M$ artworks. It is easy to calculate the optimal lower end. The total time complexity is $O(D^3(N + M))$.

Subtask 4 (16 points)

If we fix the left end and the right end, we need to solve Problem X for the y-coordinates for every artwork among the $N + M$ artworks. The total time complexity is $O(D^2(D + N + M))$.

Subtask 5 (30 points)

If we fix the left end, we decrease the right end one-by-one from b. Then, in Problem X, the following queries are given successively:

- Remove an integer from the set of integers "which should be contained in the interval."
- Calculate the minimum length of the interval satisfying the conditions.

In other words, we need to solve a dynamical version of Problem X. This can be processed in $O(1)$ time per query using a data structure such as linked list.

If we shift the left end, we need only to consider artworks which have influence. For each fixed value of the right end, each object is considered at most once. Therefore, we can solve the task in $O(D(D + N + M))$ time.

Subtask 6 (25 points)

In Subtask 5, we consider all artworks which have influence when we shift the left end. If we can only consider artworks such that the number of integers "which should be contained in the interval" is decreased by one, the total time complexity is reduced to $O(D^2 + N + M)$. This is possible if we appropriately keep track of

- the minimum value of R_j for artworks of type B satisfying $S_j = y$ for each y.

when we check all possible values of the right ends (concretely, we increase one-by-one from 0).